



PROJEKT ENGINEERING

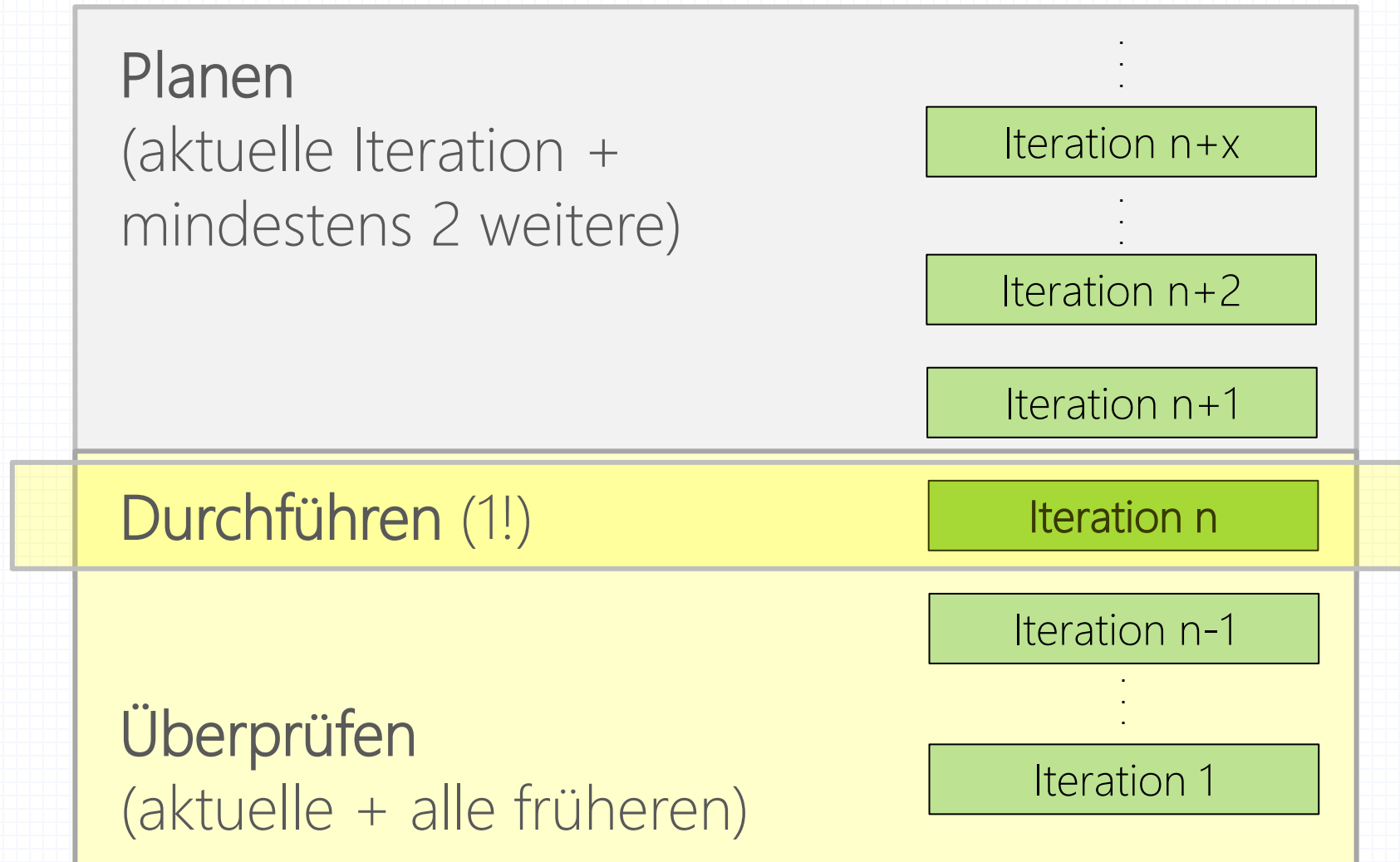
Tipps für die agile Softwareentwicklung

Herwig Mayr

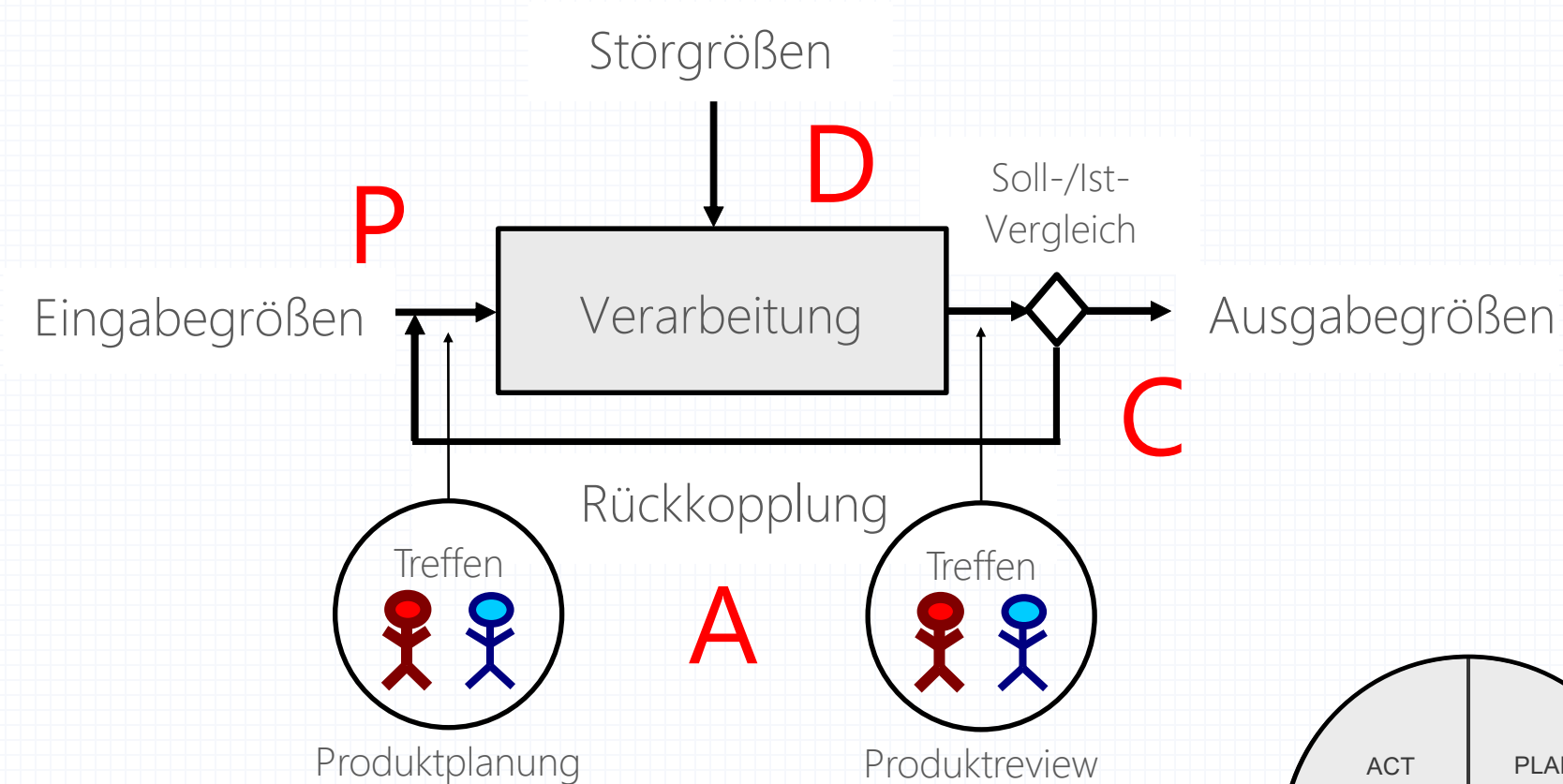
Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg

Denken in Iterationen und Zuständen

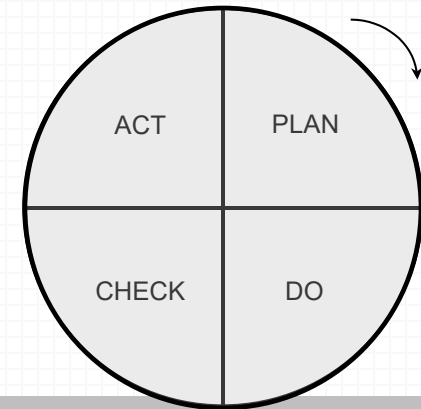
Konzept: Denken in Iterationen



Denken in Iterationen: Geregelter Prozess



Vorgehen entspricht **PDCA-Zyklus** (Deming)!

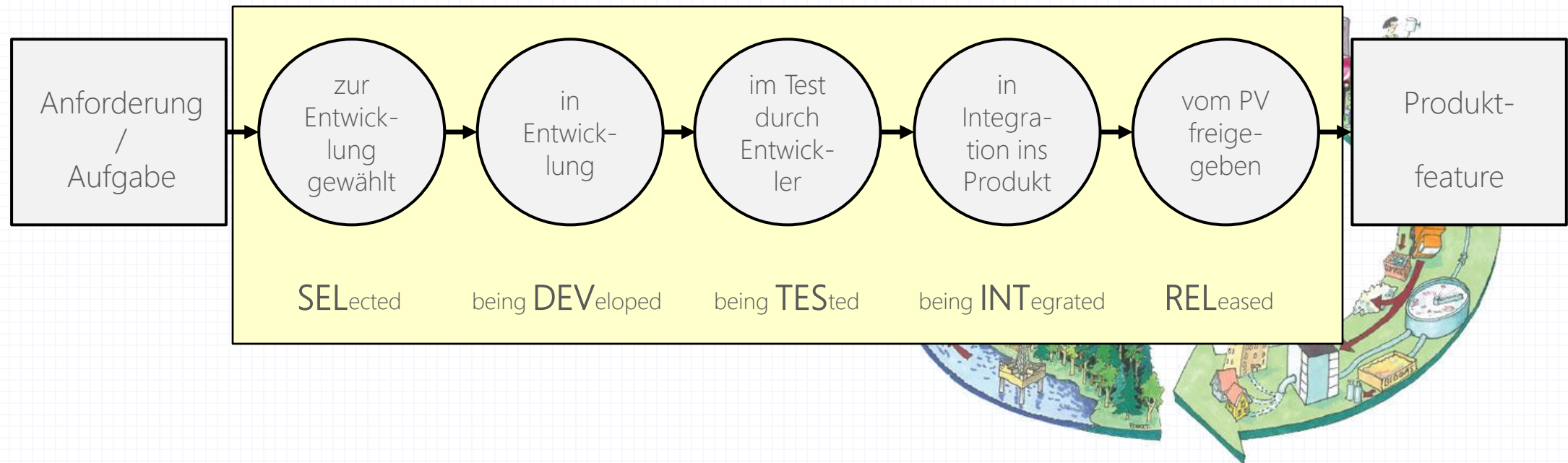


Konzept: Denken in Zuständen

Aufgaben befinden sich während der Abarbeitung in definierten Zuständen.

- Begriffe frei wählbar;
- Abfolge wichtig („Zustandskette“ bzw. „Wertschöpfungskette“)

Beispiel:



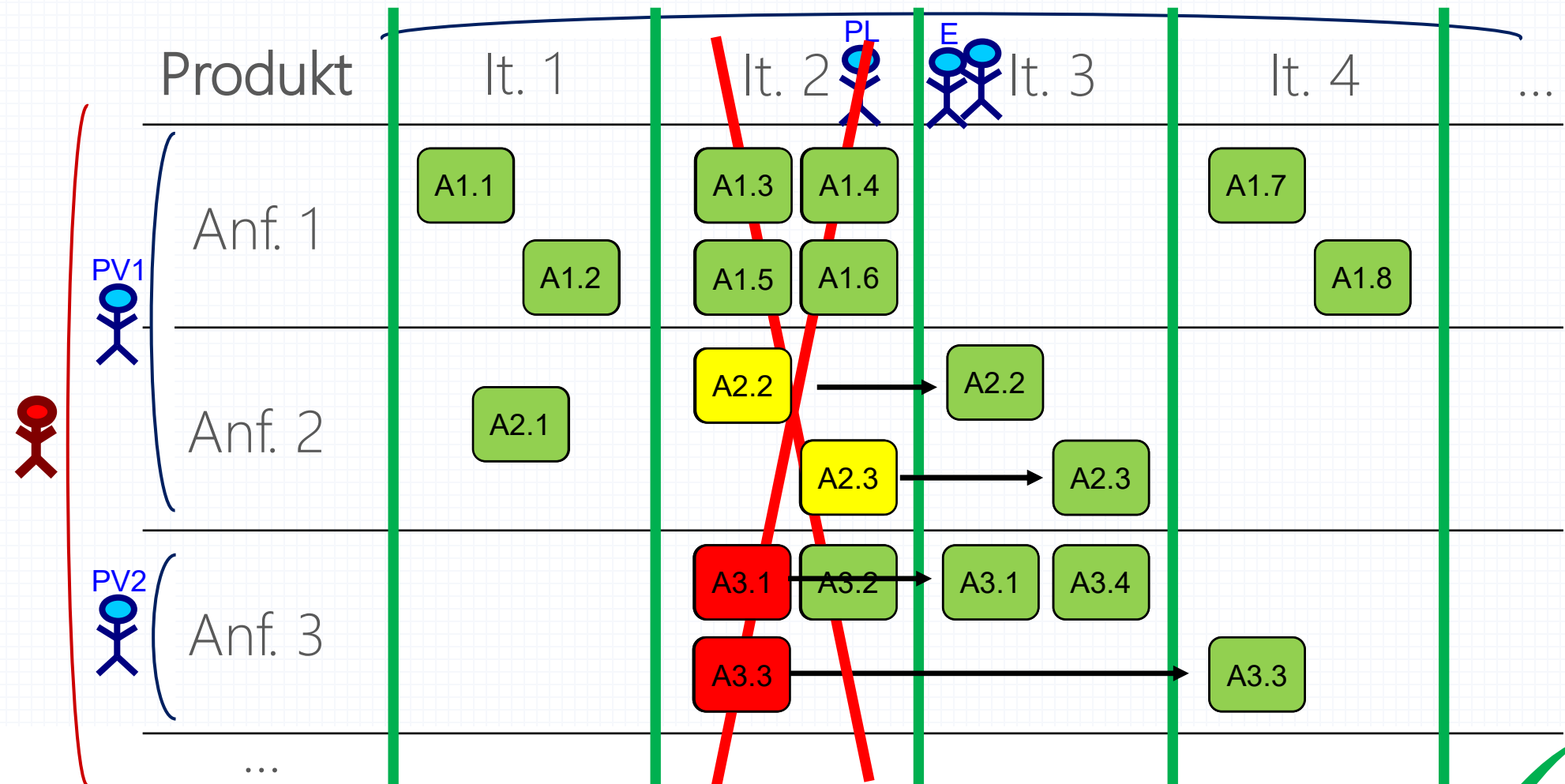
Aufgabenübersicht (Task Board)

Darstellung der **Wertschöpfungskette je Aufgabe** einer Iteration
(eine gemeinsame Übersicht pro Team!)

Anf./Aufg.	SEL	DEV	TES	INT	REL
Anforderung 2					
Aufgabe 2.1					
Aufgabe 2.3					
Aufgabe 2.4					
Anforderung 3					
Aufgabe 3.3					
Aufgabe 3.4					
Anforderung 5					
Aufgabe 5.1					
Aufgabe 5.2					
...					

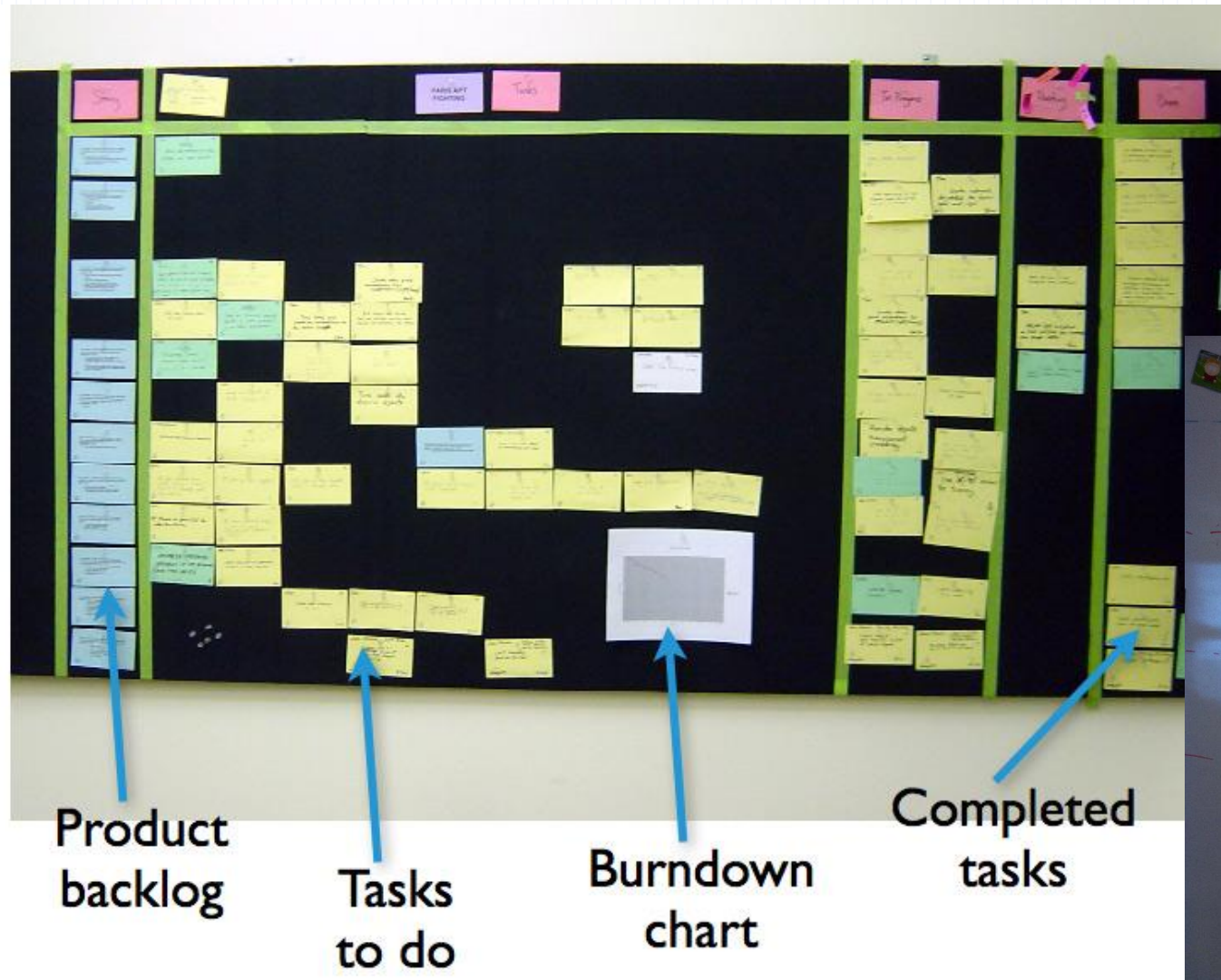
Festgelegte Regeln für den Spaltenübergang sind wichtig („**Definition of Done**“)!

Anforderungen/Aufgaben versus Iterationen



1. Plane für eine Iteration Aufgaben im geeigneten Umfang.
2. Schließe zumindest die Aufgaben einer Anforderung je Iteration ab.
3. Schließe nicht abgeschlossene Aufgaben in der nächsten Iteration ab.
4. Entscheide für jede abgebrochene Aufgabe, ob und in welcher Iteration sie fortgeführt wird.

Beispiel: Scrum Task Board



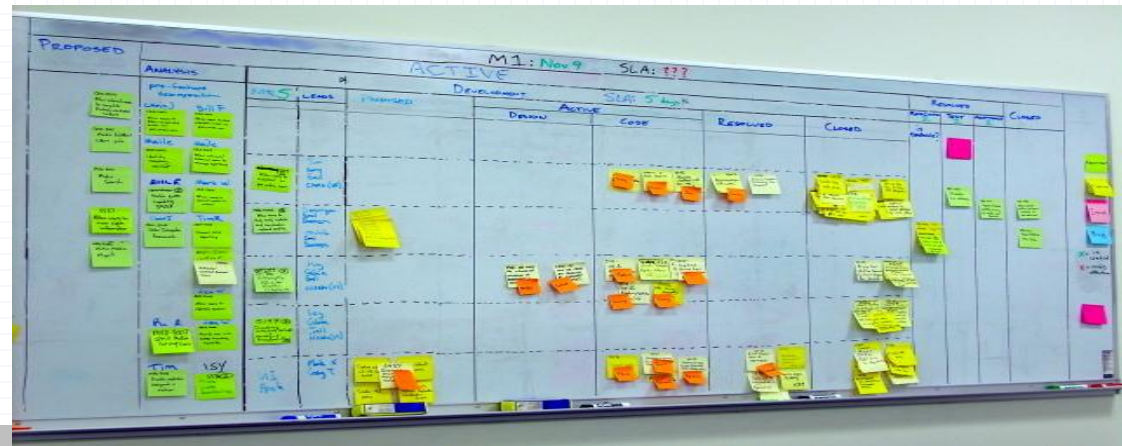
Abarbeiten von Zustandsketten: Kanban

Kanban: von japanisch **Kan** = Signal; **Ban** = Karte

- nach dem 2. Weltkrieg in Japan erfunden (T. Ohno, Toyota)
- Produktablaufsteuerung zur Optimierung des Lagerstands
- betont das Pull-Prinzip (Zulieferung je nach Verbrauch)



von D. Anderson **für IT-Projekte adaptiert** und um Konzepte der agilen Entwicklung erweitert (2007)



(Grafik: <http://theindependentcharacters.com>)

Kanban-Idee: Beispiel Essenszubereitung



Kanban: Abgrenzung

KEIN (agiles) Vorgehensmodell, sondern Entwicklungstechnik!

(aber beim agilen Vorgehen einsetzbar, z.B. in Scrum -> „Scrumban“)

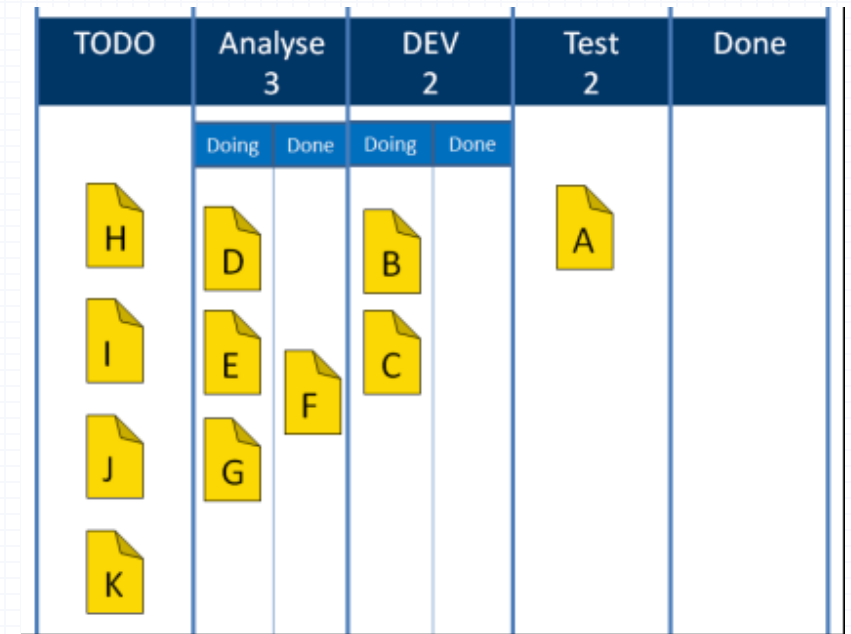
- kein Requirements Management
- keine Teamorganisation, keine Rollen
- keine zeitlichen Vorgaben / Durchlaufzeiten -> keine Terminplanung
- keine Iterationen (!), außer in Notfällen, aber inkrementell (falls sinnvoll)
- Releasezyklen frei wählbar
- Synchronisation der Aufgabenerledigung möglich
- auch für mehrere Projekte zur selben Zeit!

-> Gut einsetzbar für kleine, kurz getaktete, möglichst unabhängige Aufgaben (z.B. Wartungstätigkeit)!

Kanban: Empfehlungen

nach D. Anderson:

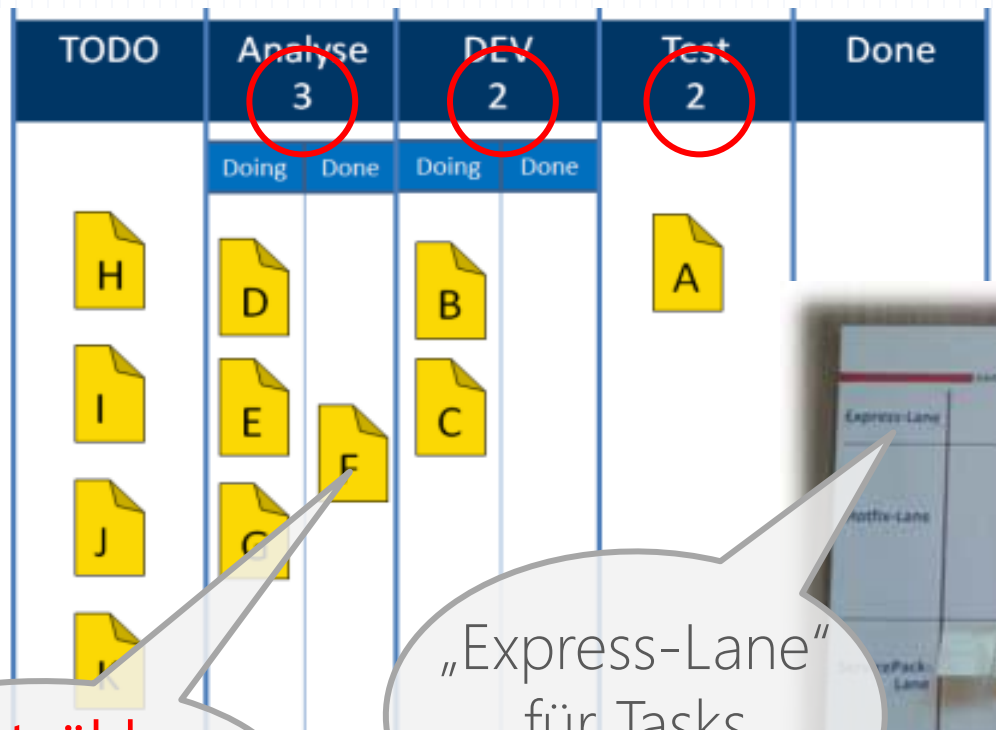
- Visualisiere den Fluss der Arbeit.
(Kanban-Board)
- Begrenze die Menge angefangener Arbeit.
(Ticket System, Pull-Prinzip)
- Miss und steuere den Fluss.
(Cycle Time Analysis, „Cost of Delay“)
- Mache die Regeln für den Prozess explizit.
(Definition of Done)
- Verwende Modelle, um Chancen für kollaborative Verbesserungen zu erkennen.
(Kaizen)



Darstellung am Kanban-Board

Anzahl der *Elemente pro Spalte* ist *begrenzt* (außer Eingang + Ausgang)!

-> Zahl in Spaltenheader; Theory of Constraints



Mehrere Teams können
sich ein Board teilen!

„Swim Lanes“ (oder
versch.farbige Kärtchen)
für Teilprojekte

„Express-Lane“
für Tasks
vom Chef ☺

Nichtzählen
problematisch!



(Bild: Fa. eurofunk Kappacher, St. Johann/P.)

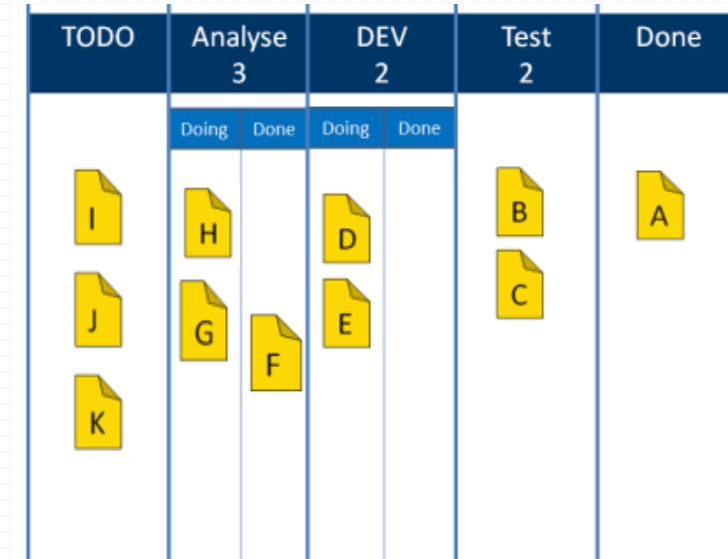
Kanban-Board: Fortschritt und Zielerreichung

Messung des Fortschritts:

- durch „Cycle Time Analysis“
- Zweck: gleichmäßige Arbeitsbelastung (vgl. „40 hour-week in XP“, „regular work load“ in Scrum)
- Notwendigkeit: ~ gleich große Aufgaben (oft schwierig zu erreichen bzw. vorab schwer schätzbar)
- Nur dann ist die Zykluszeitmessung aussagekräftig! („Cycle Time“ beschreibt, wie lange Element x in Spalte y ist.)

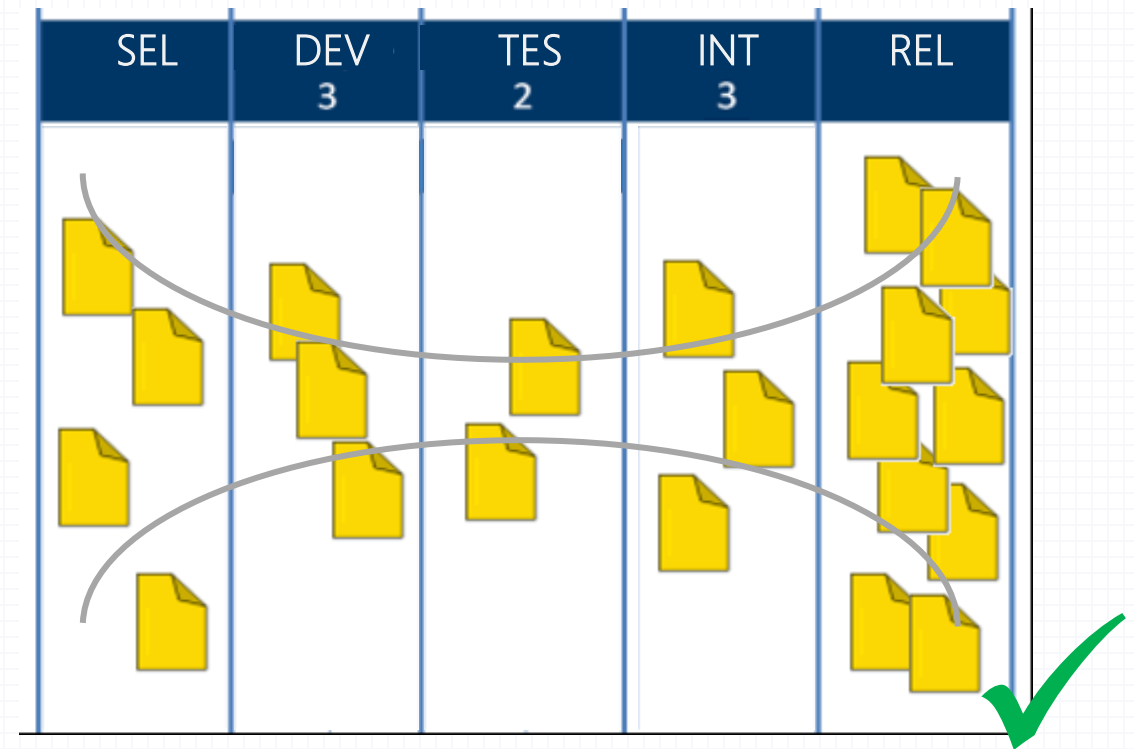
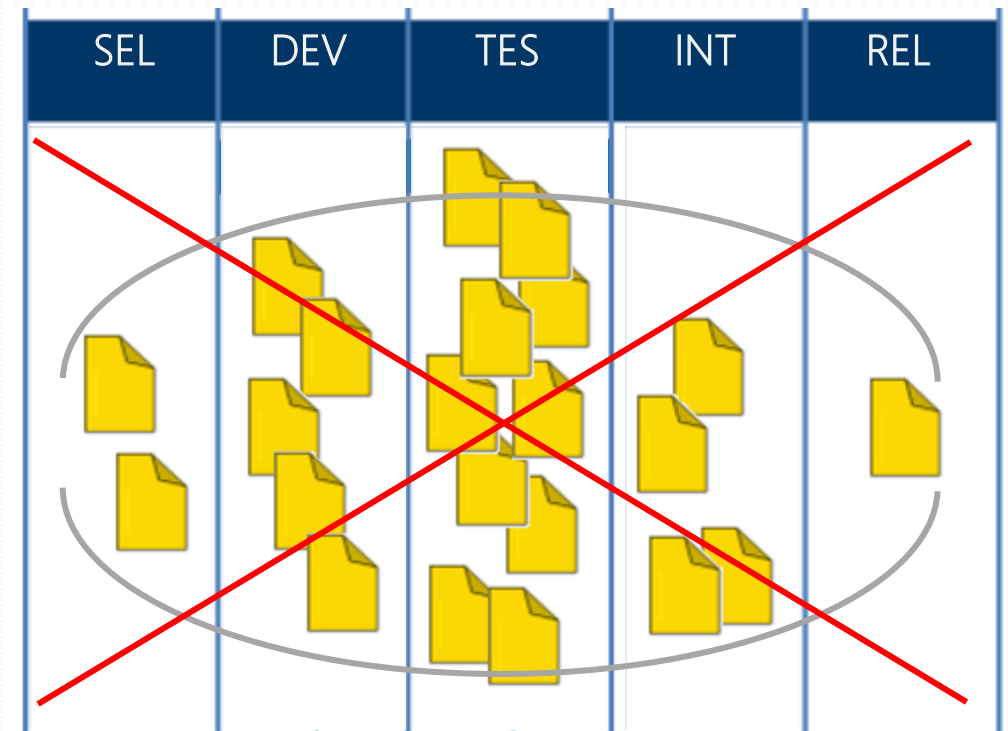
Messung der Zielerreichung:

- durch Anzahl der Elemente (**erledigte Aufgaben**) in der rechten Spalte („Released“; „Done“) – auf **Zuordnung zu Anforderungen** achten!



Kanban: Hauptnutzen

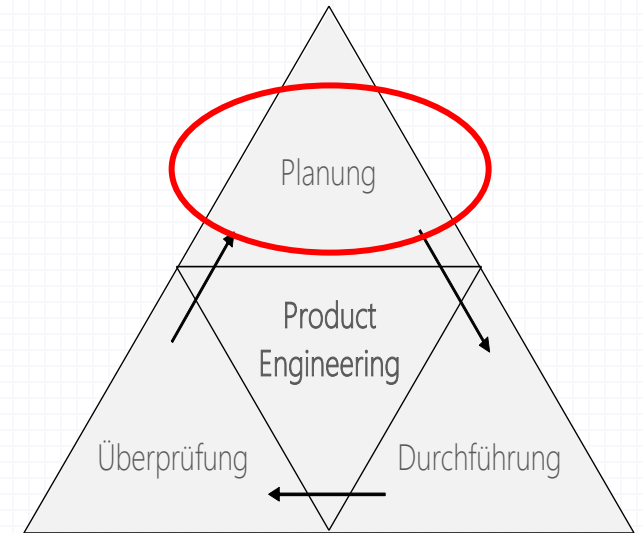
Kanban Flow (am Kanban-Board): Grafische Visualisierung von **Fortschritt** und **Zielerreichung**:



Tipps zum (agilen) Planen

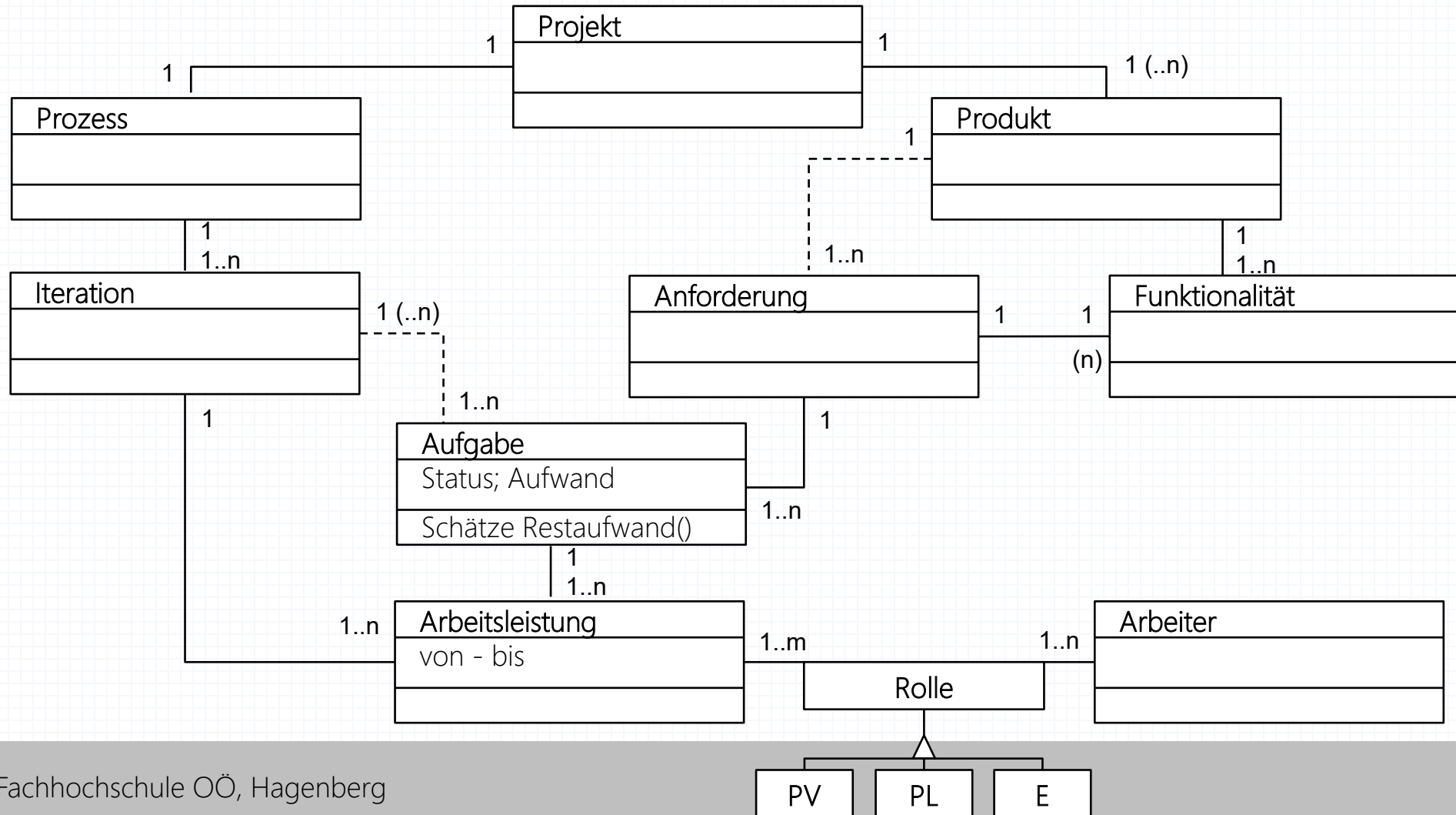
Was wird beim agilen Vorgehen geplant?

- **Entwicklung:** Struktur und Abläufe
- **Ergebnis:** Benutzung, Anwendungsfälle
- **Prozess:** (Um-)Planen aktueller/zukünftiger Iteration(en)



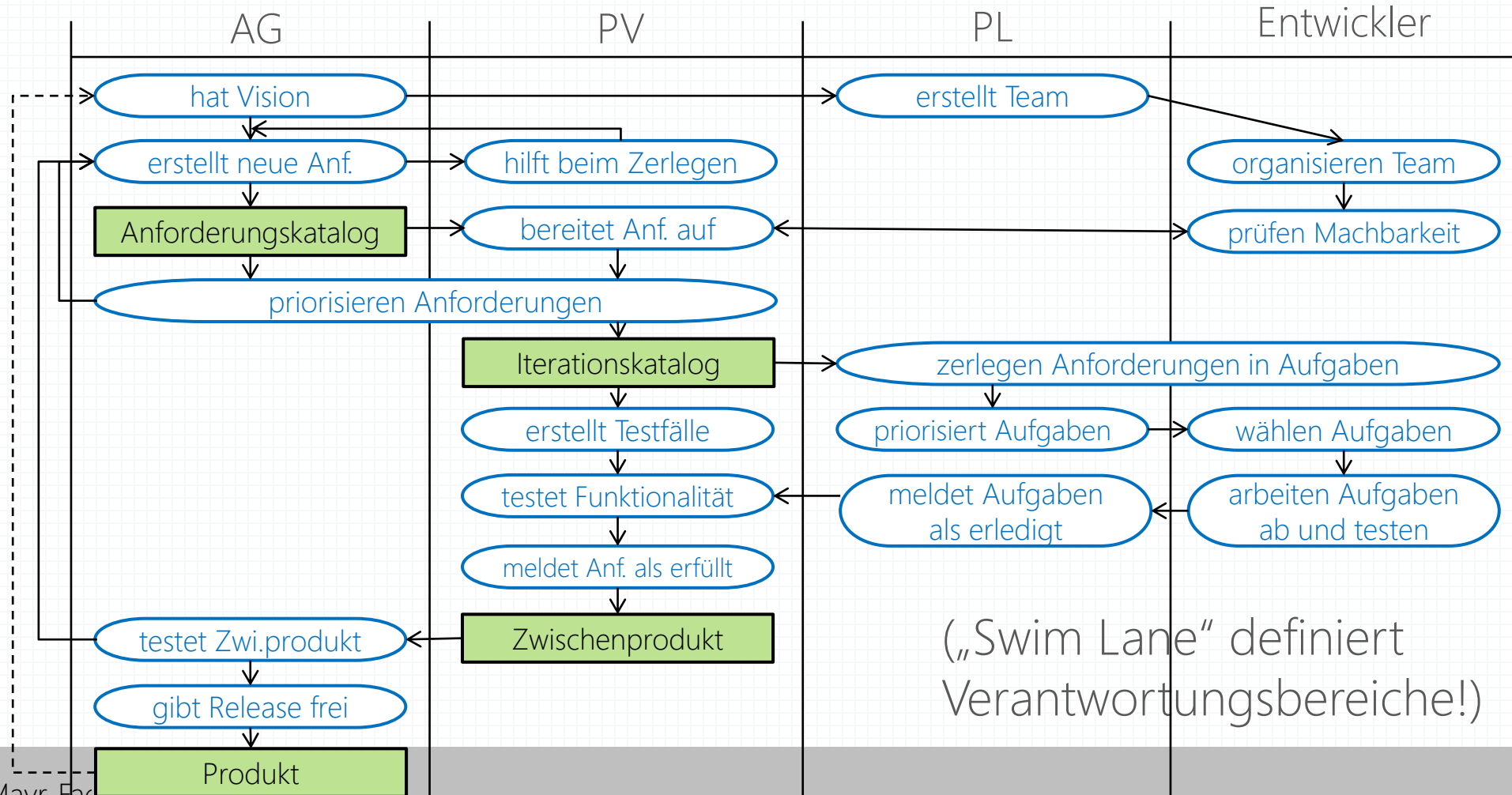
Planen: Statische Zusammenhänge (II)

Klassendiagramm am Beispiel „Projekt“:



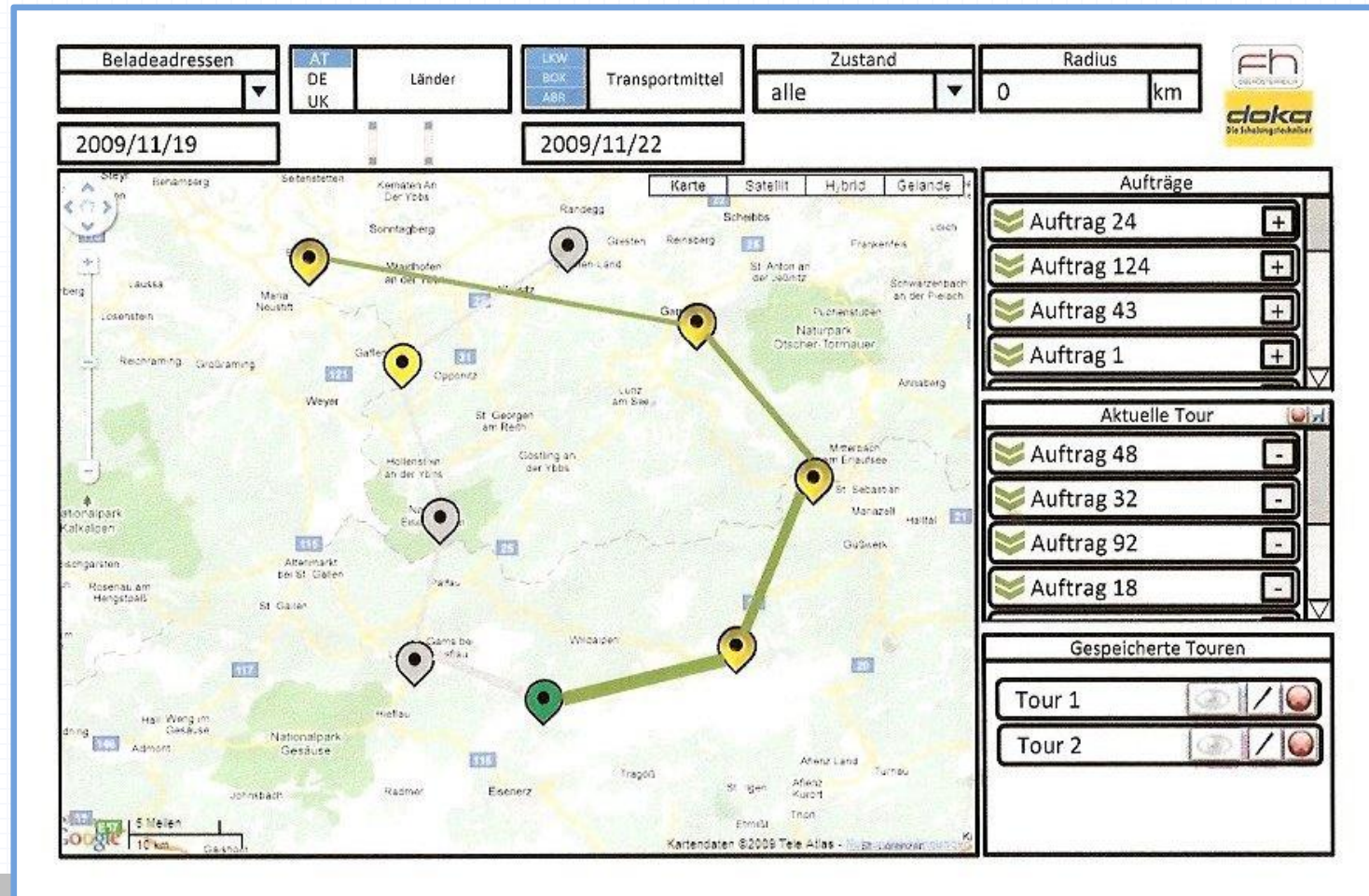
Planen: Dynamische Zusammenhänge

Abläufe gut darstellbar als (UML-)Schwimmbahndiagramm:



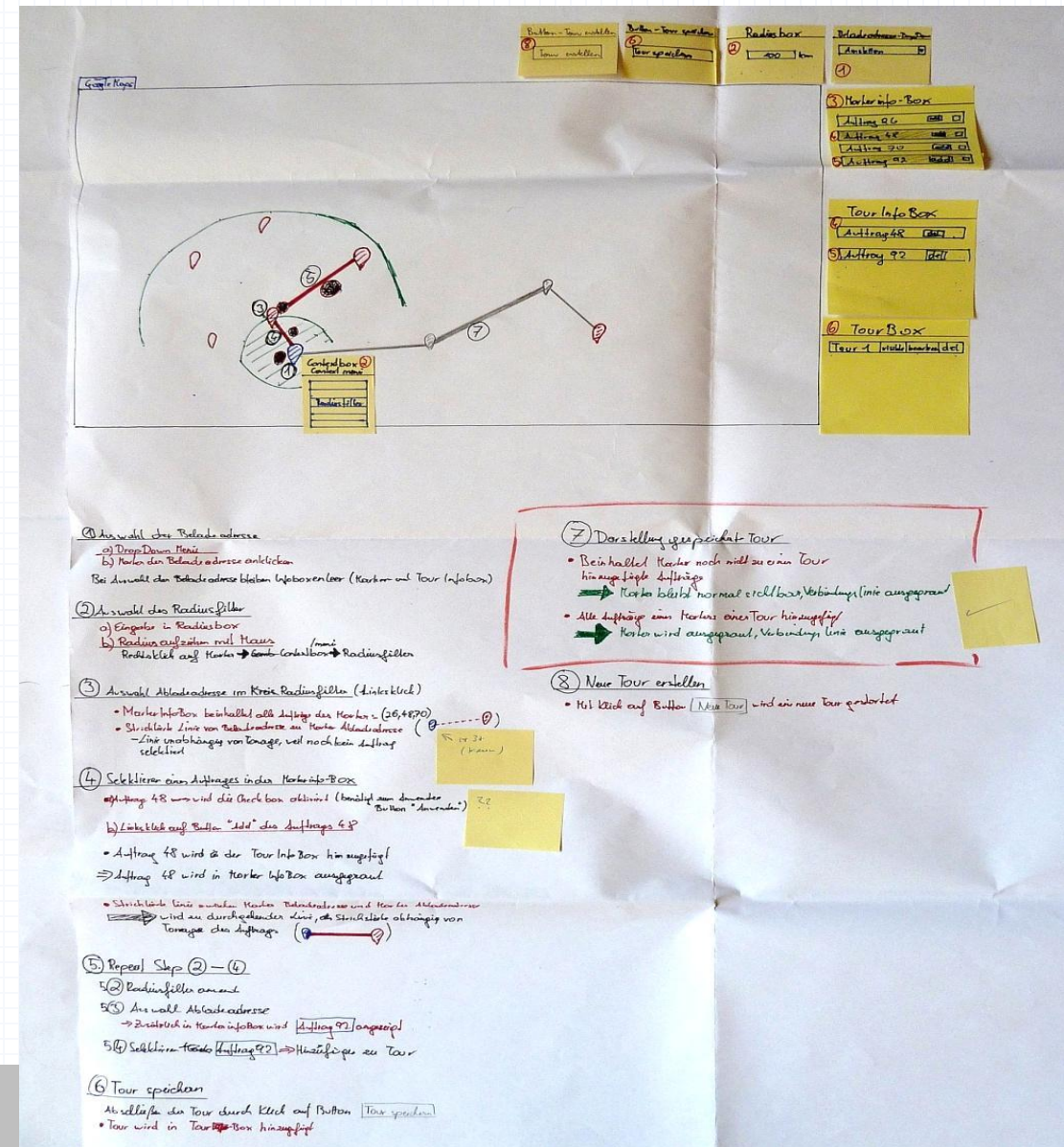
Planen: Erarbeiten der Ablauflogik (II)

Mittels **Benutzerschnittstellen-Prototyp** (später als Software):



Planen: Erstellen typischer Anwendungsfälle (I)

Diese geben die Reihenfolge der Aufgabenrealisierung vor!



Planen: Erstellen typischer Anwendungsfälle (II)

① Auswahl der Beladeadresse

- a) Drop Down Menü
- b) Markieren der Beladeadresse anklicken

Bei Auswahl der Beladeadresse bleiben Infoboxen leer (Marker und Tour Info Box)

② Auswahl des Radiusfilter

- a) Eingabe in Radiusbox
- b) Radius aufziehen mit Maus

Rechtsklick auf Marker → Kontextbox → Radiusfilter

③ Auswahl Abladeadresse im Kreis Radiusfilter (Linksklick)

- Marker Info Box beinhaltet alle Aufträge des Markers (26,48,70)
- Strichlinke Linie von Beladeadresse zu Marker Abladeadresse
- Linie unabhängig von Tour, weil noch kein Auftrag selektiert

④ Selektieren eines Auftrages in der Marker Info Box

- a) Auftrag 48 wird in der Tour Info Box hinzugefügt
- b) Linksklick auf Button "Add" des Auftrags

- Auftrag 48 wird in der Tour Info Box hinzugefügt
- ⇒ Auftrag 48 wird in Marker Info Box ausgeprägt

- Strichlinke Linie zwischen Marker Beladeadresse und Marker Abladeadresse
- wird zu durchgehender Linie, da Strichstärke abhängig von Tour des Auftrags

⑤ Repeat Step ② - ④

5② Radiusfilter anpassen

5③ Auswahl Abladeadresse

→ Zusätzlich in Marker Info Box wird Auftrag 92 ausgewählt

⑦ Darstellung gespeicherter Tour

- Beinhaltet Marker noch nicht zu einer Tour hinzugefügte Aufträge
→ Marker bleibt normal sichtbar, Verbindungslinie ausgeprägt
- Alle Aufträge eines Markers einer Tour hinzugefügt
→ Marker wird ausgeprägt, Verbindungslinie ausgeprägt

⑧ Neue Tour erstellen

- Mit Klick auf Button "Neue Tour" wird eine neue Tour gestartet

Produktverantwortlicher erstellt Anwendungstests gemäß dem Ablauf der Anwendungsfälle!

Planen: Interne versus externe Anforderungen/Aufgaben

Festlegung der für den Auftraggeber sichtbaren Ergebnisse:

- **Externe Anforderungen/Aufgaben:** führen zu einem Ergebnis mit unmittelbarem Wert für den AG
- **Interne Anforderungen/Aufgaben:** dem AG nicht direkt „verkaufbar“, aus Projektsicht (AN) aber notwendig
- Einteilung über Projektverlauf einheitlich halten (**Checkliste**)
- Einheitlichkeit notwendig für Vergleiche zwischen Projekten (⇒ **Management-Overhead!**)

Planen: Sonderfall Forschungsaufgabe

“Research Task” oder “Spike”: Ergebnis für Projekt wichtig, aber Erreichungswahrscheinlichkeit und -grad unsicher

Vorgehen:

- Anlegen einer Aufgabe **ohne** Zeitschätzung
- Zuordnung von Mitarbeitern
(Jeder Mitarbeiter darf **max. eine** Forschungsaufgabe gleichzeitig bearbeiten!)
- besondere Berücksichtigung in der Risikobehandlung
- laufendes Buchen des Aufwands auf die Aufgabe
- täglicher Bericht über den Status der Aufgabe durch Mitarbeiter
- jederzeit Abbrechen der Aufgabe durch den Produktverantwortlichen möglich („Plan B“ ist sinnvoll)



Planen: Umplanen der aktuellen Iteration

Gründe für kurzfristige Umplanungen:

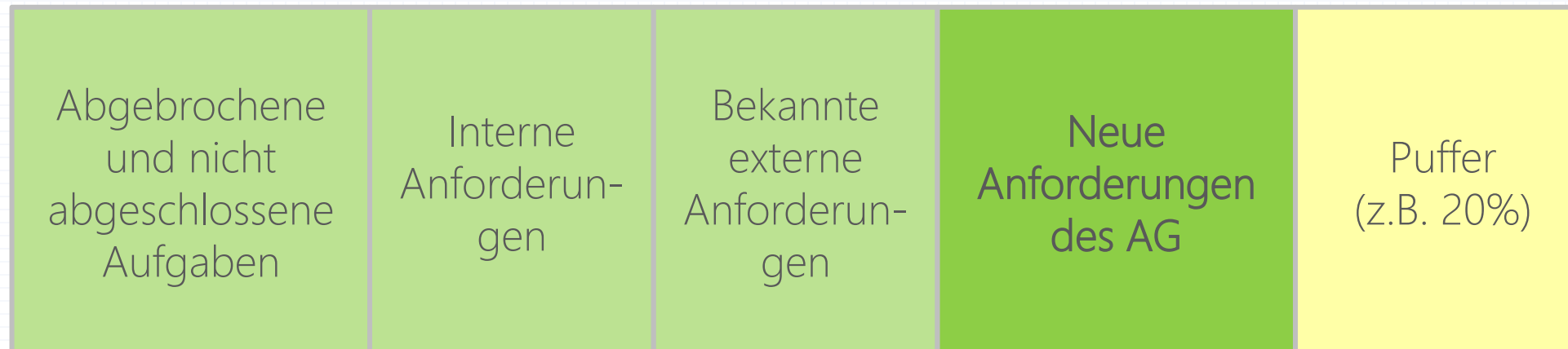
- Mitarbeiter schließen Aufgaben früher als erwartet ab (priorisierte Liste weiterer Aufgaben vorhalten!)
- Aufgaben werden abgebrochen („Plan B“ soll vorhanden sein)
- Aufgaben werden obsolet (vom AG nicht länger benötigte Funktionalität)
- Störgrößen beeinflussen den Projektverlauf (Krankheit, Marktveränderungen, ...)



(Bild: en.wikipedia.org)

Planen: Vorplanen zukünftiger Iterationen

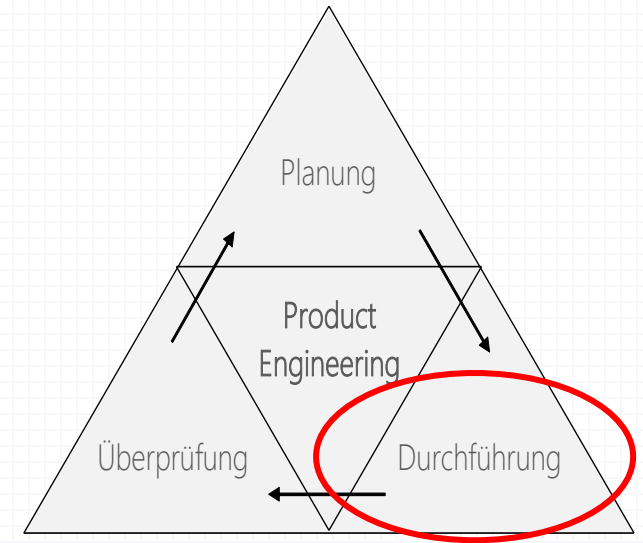
Teilbereiche:



Tipps zum (agilen) Durchführen

Tipps zum Durchführen

- Arbeitszeit erfassen
- Restaufwand schätzen
- Aufgabenstatus aktualisieren



Durchführen: Erfassen der Arbeitszeit

Prinzipien:

- Alle auf das Projekt zu buchende Arbeit wird erfasst (evtl. internes und externes Projektlogbuch).
- Jede Arbeit(seinheit) wird einer Aufgabe zugeordnet (ggf. **vorher** Aufgabe erstellen!).
- Jeder Mitarbeiter erfasst jeden Tag seine Arbeit.
- Eine sinnvolle kleinste Einheit ist zu definieren (z.B. 0,5 h).



(Bild: www.linuxfibel.de)

Durchführen: Schätzen des Restaufwands

Jeden Tag erneut; Wert kann auch erhöht werden!

 **Projekt - Durchführung - 3. Iteration - 11. Kriterienbewertung am PDA - 11.1. Sourcecode dokumentieren**

Geleistete Arbeitszeit eintragen

Aufgabe

11.1. Sourcecode dokumentieren

Verantwortlicher Pilotanwender

carmen

Mitarbeiter

thomas

Iterationstag

02.06.2005

letzte Schätzung

UE

wird später ev. nicht angezeigt

geleistete Arbeitseinheiten

UE

neue Schätzung

UE

Änderungen übernehmen

Änderungen verwerfen

Aufgabe abschließen

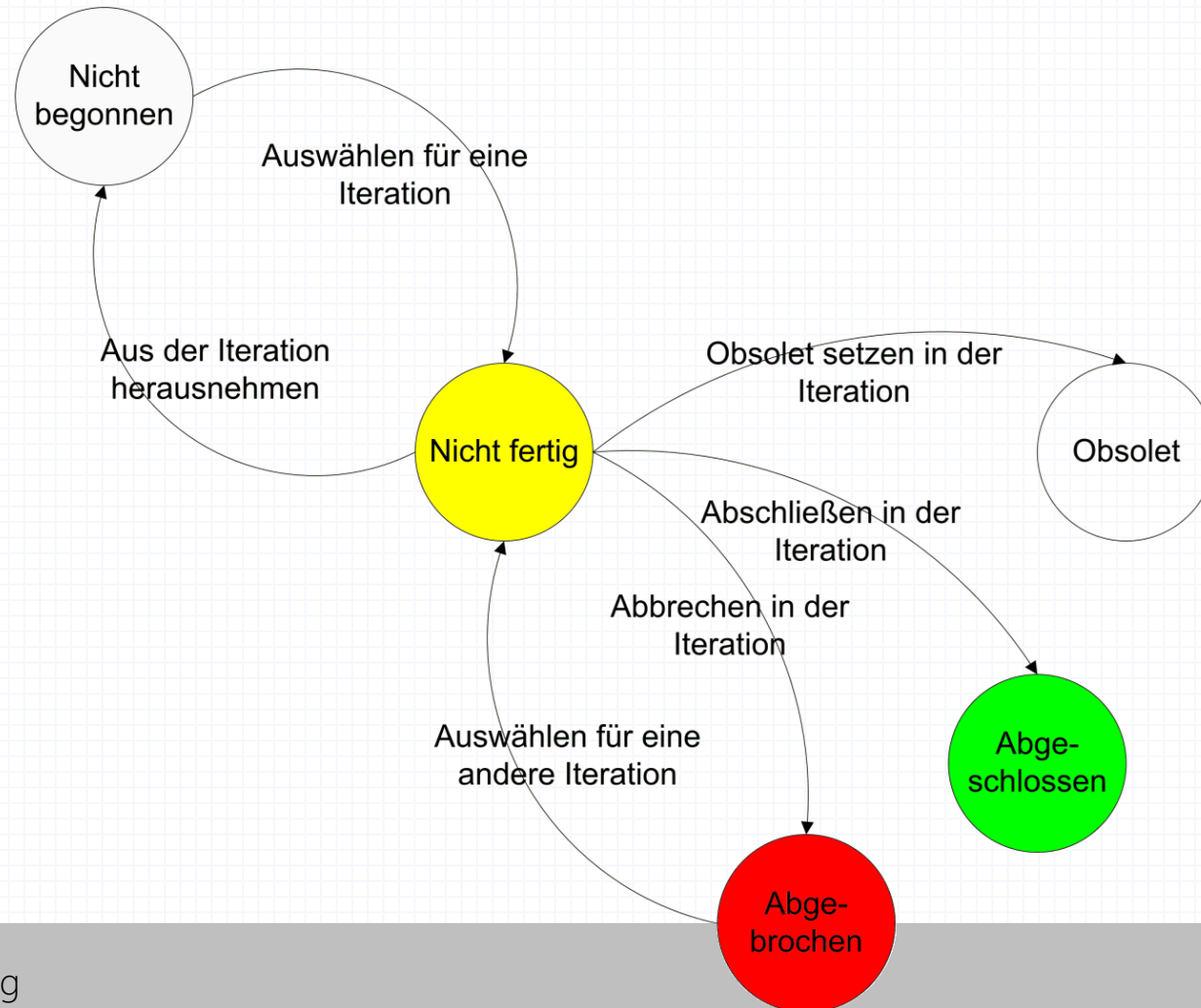
Aufgabe abbrechen

Aufgabe obsolet setzen

(mit oder ohne Vorgabe
der Letztschätzung)

Durchführen: Ändern des Zustands von Aufgaben ("Status")

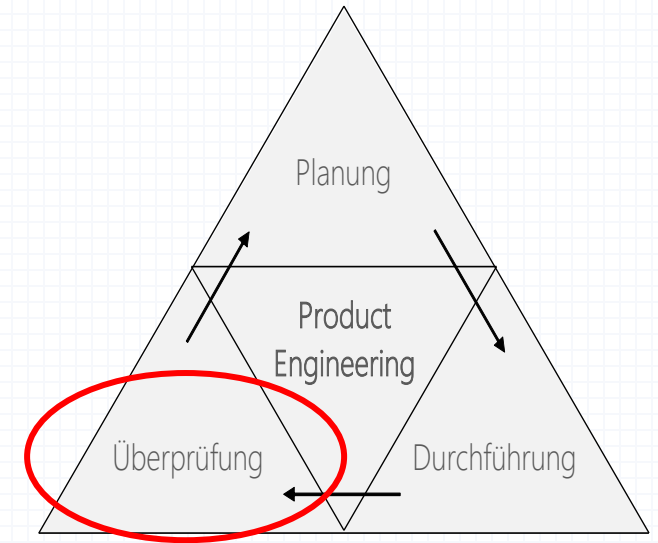
Möglichst mittels **Tracking-System** und **Tickets**:
(Ampel-Metapher!)



Tipps zum (agilen) Überprüfen

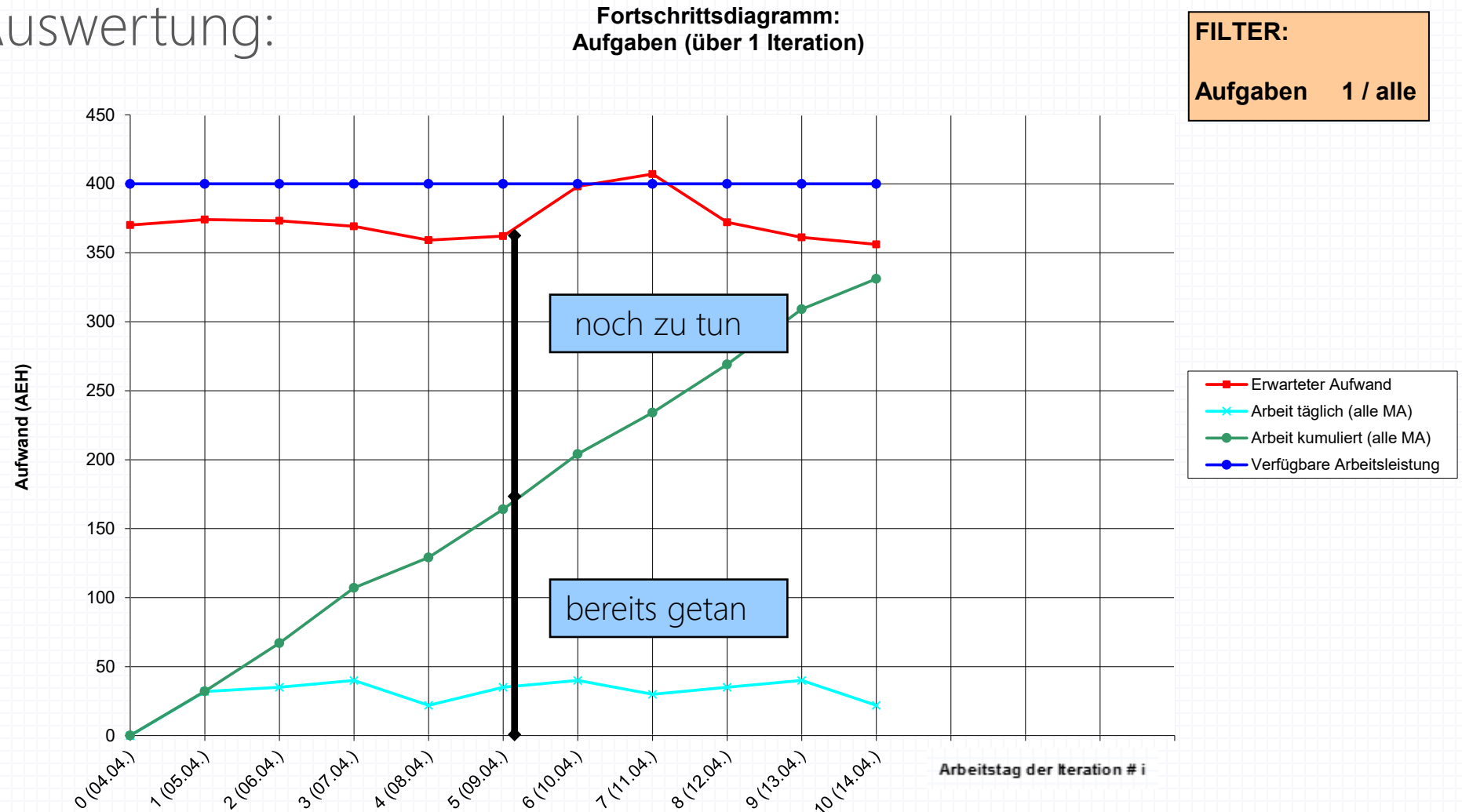
Tipps zum Überprüfen

- Fortschritt und Zielerreichungsgrad feststellen
- Management-Overhead analysieren
- Mitarbeiterbelastung & Schätzqualität feststellen
- Prozess der Softwareentwicklung verbessern



Überprüfen: Aufgaben – Fortschritt / Zielerreichung

Grafische Auswertung:



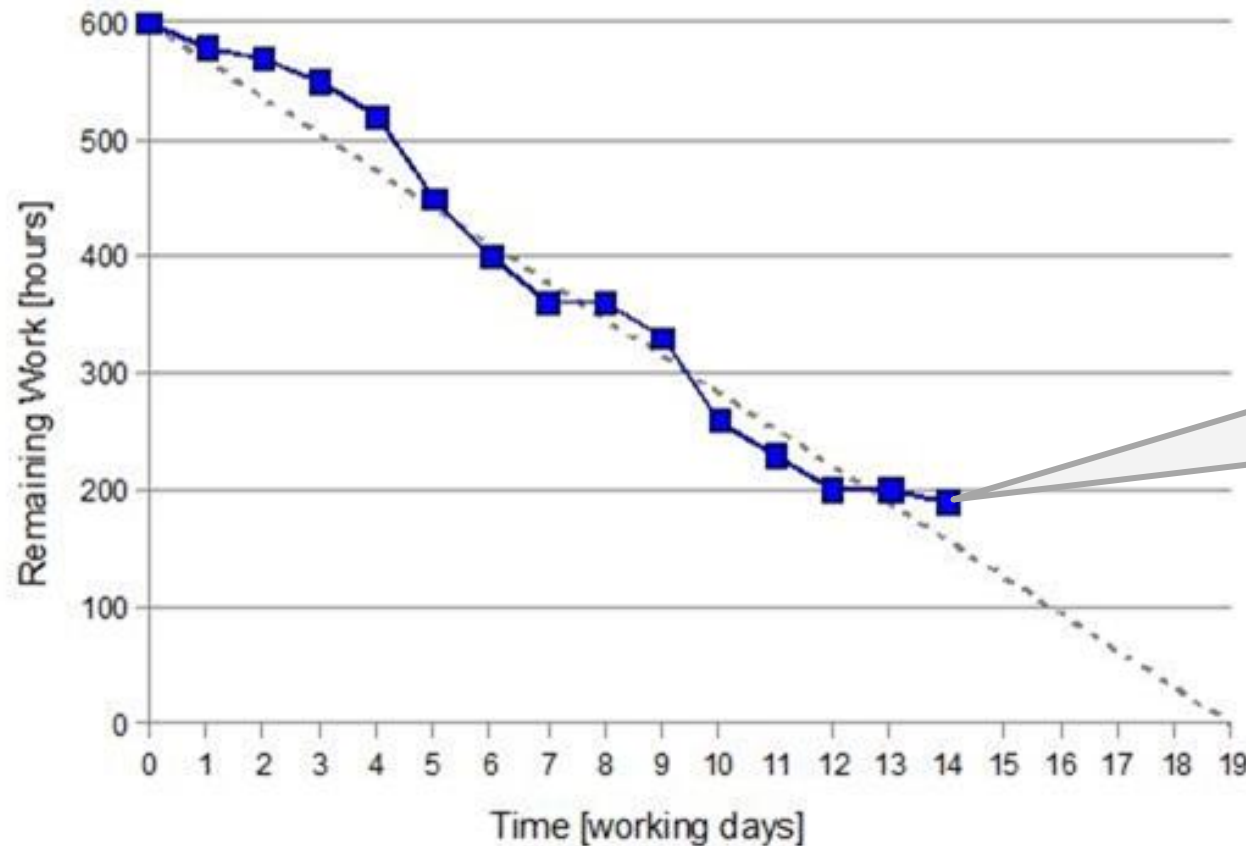
Überprüfen: Aufgaben – Fortschritt / Zielerreichung

Gliederung nach Anforderungen für AG-Sicht:
(mind. 1 Anforderung pro Iteration erfüllt, Ampel-Metapher!)

Projekt - Iterationen - 1. Iteration	
Status	Nummer
7 Erweiterung des Datenmodells	1
7.1 Vereinsnummer hinzufügen	Arbeitstage gesamt
7.2 Bezeichner für Ergebnisse in Datenmodell einfügen	7
7.3 Datenmodell diskutieren	Beginn
1 Aspektbewertung am PDA	28.04.2005
1.1 Kriterien- und Aspektbewertung mit Punktegenerator verbinden	Ende
1.2 Formular für die Aspekte erstellen	13.05.2005
1.3 Aspektformulardaten speichern	Voraussichtlicher Gesamtaufwand
1.4 Aspektformulardaten auslesen	211
4 Punktegenerator	- Bisheriger Aufwand
4.1 Logik überlegen	193
4.2 Logik implementieren	Offener Restaufwand
2 Generierung eines Gesamtberichts	18
2.1 Testdaten erstellen	Verfügbare Arbeitszeit
2.2 Gesamtbericht testen	168
2.3 Vorhandenen Gesamtbericht bearbeiten	- Voraussichtlicher Gesamtaufwand
	211
	Reserve
	-43

Überprüfen: Anforderungen– Fortschritt / Zielerreichung

Grafische Auswertung (I): **Sprint Burndown Chart** (1 Iteration; aus Scrum)

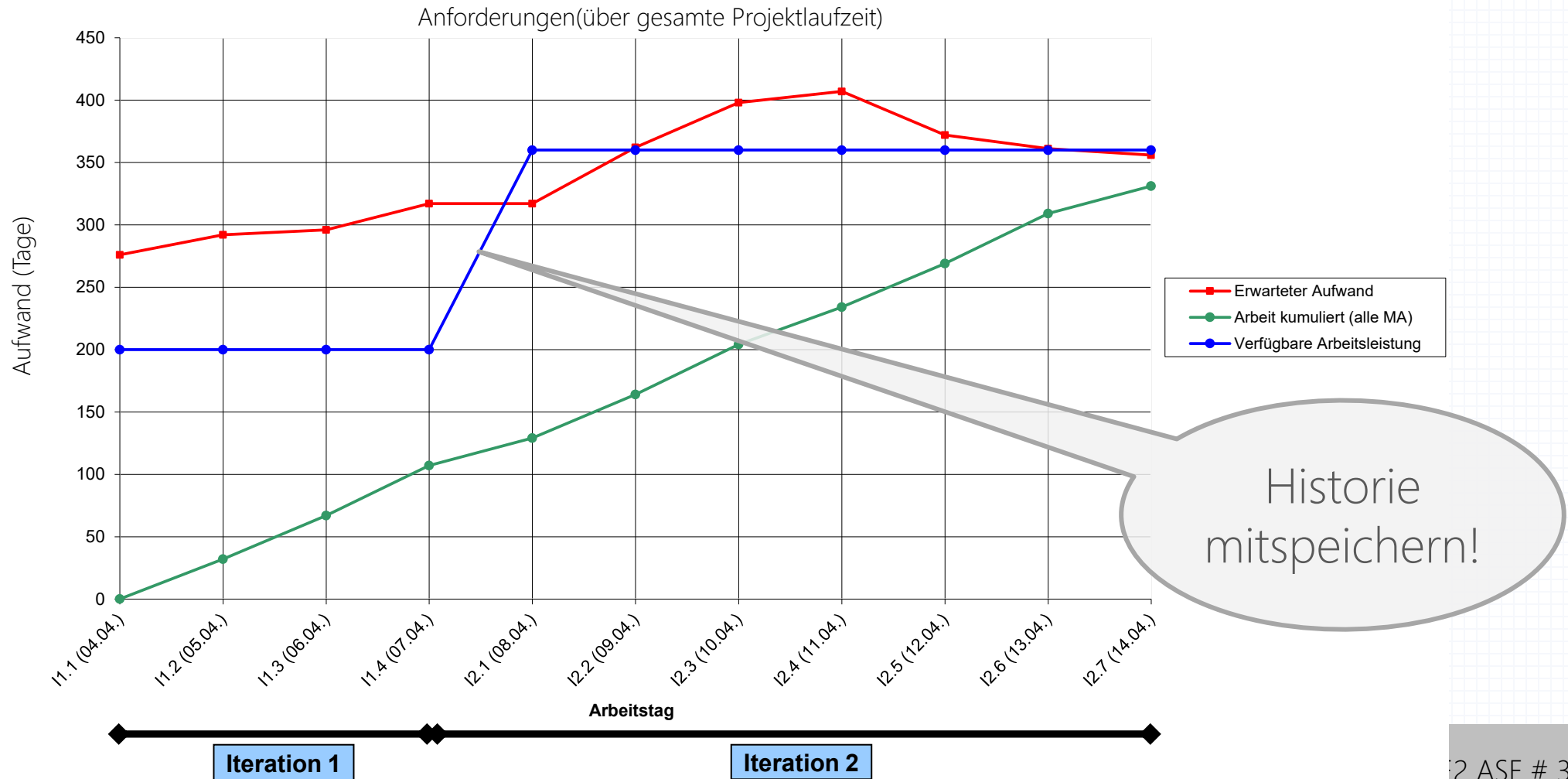


(Grafik: <http://www.sw-engineering-candies.com>)

Nur geschätzter
Restaufwand; -
geleistete Arbeit
wird **nicht** erfasst!

Überprüfen: Anforderungen– Fortschritt / Zielerreichung

Grafische Auswertung (II): Fortschrittsdiagramm (mehrere Iterationen)



Überprüfen: Management-Overhead (I)

Ermittlung des **Management-Overhead** (= Verhältnis zwischen internen und externen (bzw. gesamten) **Aufwänden** in Projekt)

Zweck: Darstellung des Management-Overhead als

1. notwendig
2. begründbar
3. in der Höhe akzeptabel

Arten:

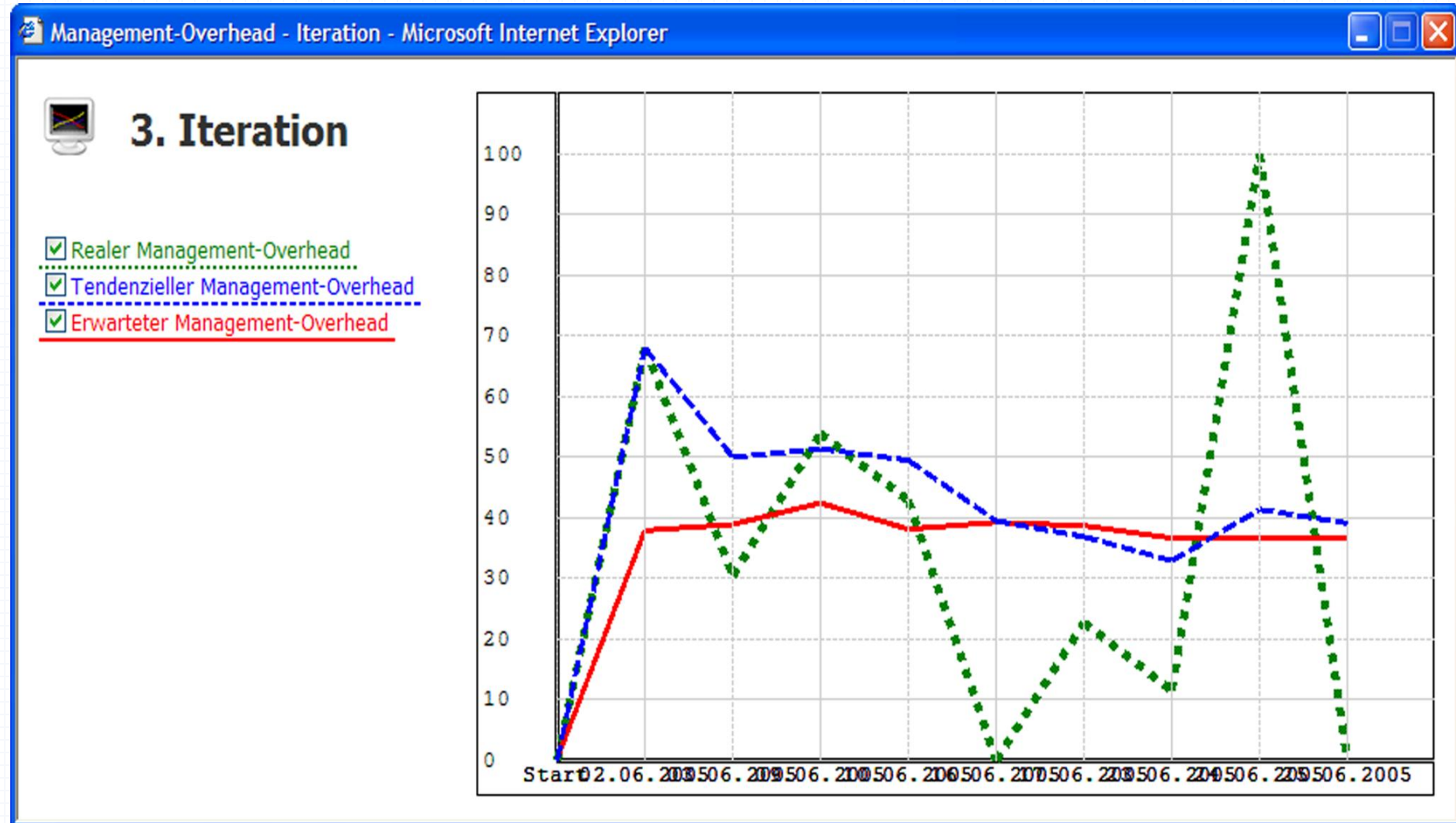
- **realer Management-Overhead** (Gegenwart)
- **tendenzieller Management-Overhead** (Vergangenheit)
- **erwarteter Management-Overhead** (Vergangenheit + Zukunft)



(Bild: <http://www.grandviewbusinesssolutions.com>)

Überprüfen: Management-Overhead (II)

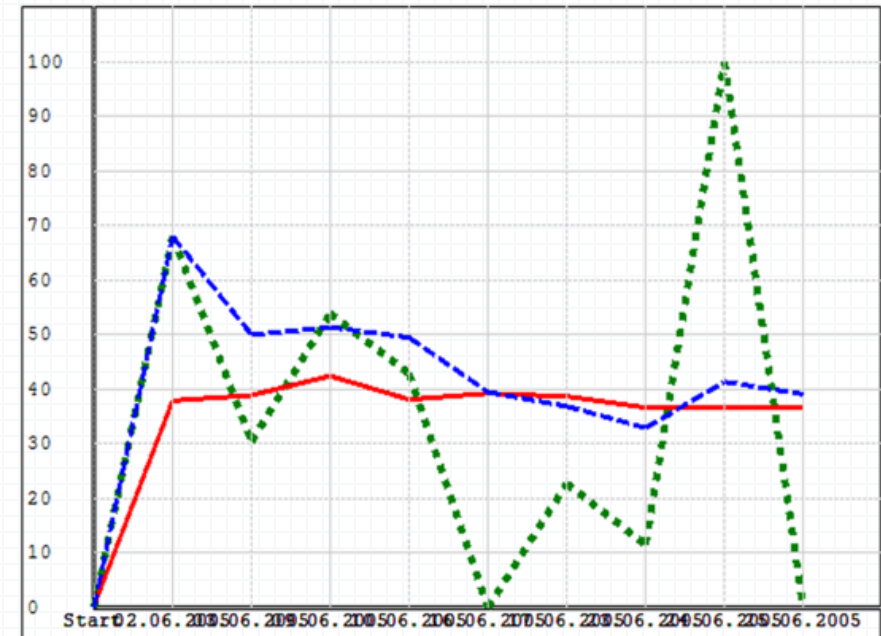
Beispiel:



Überprüfen: Management-Overhead (III)

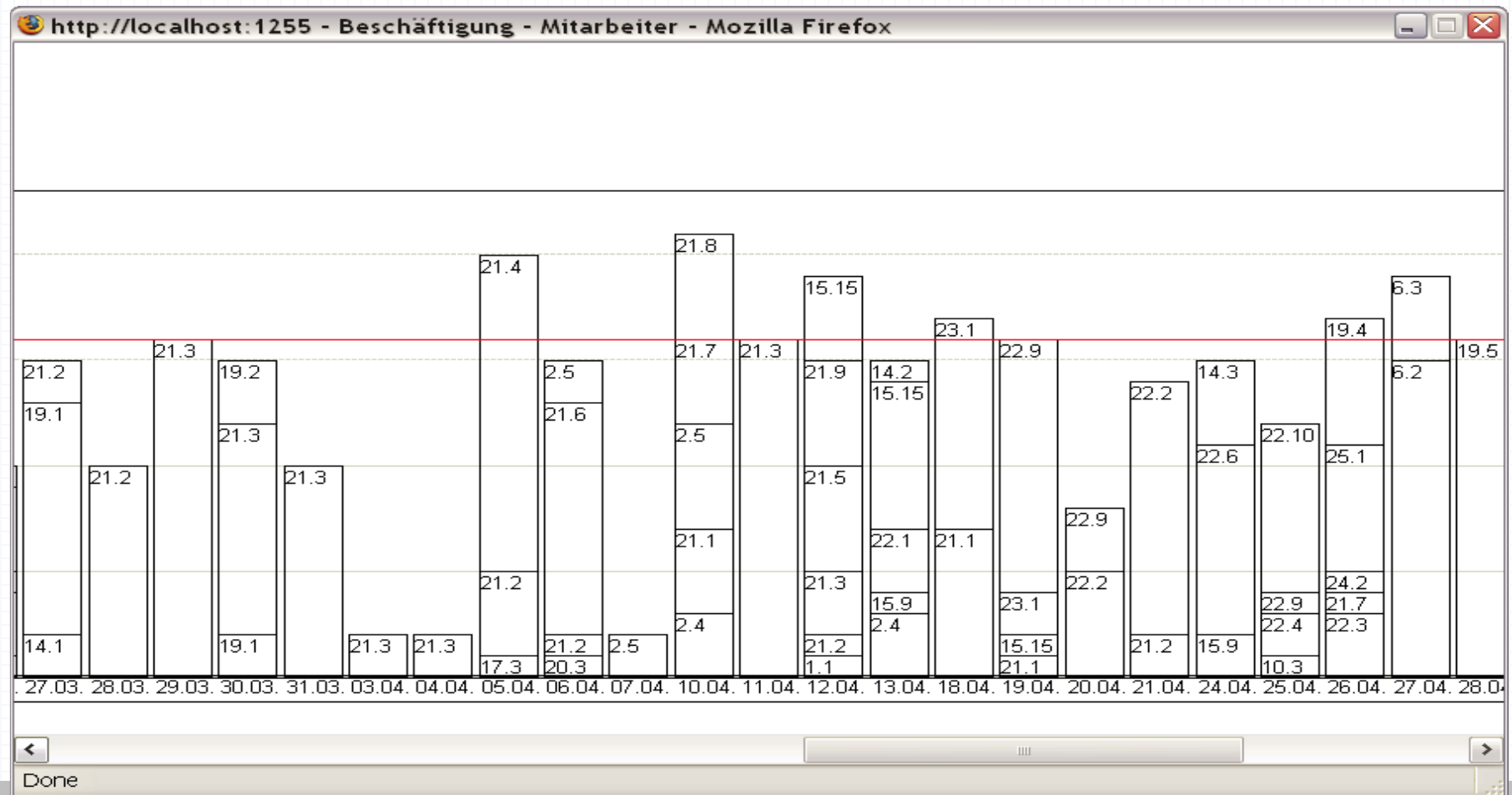
Vorteile eines explizit ausgewiesenen Management-Overhead:

- leichteres Erkennen von „overmanaged“ Projekten
- präziser ermittelbare Aufschläge
- genauere Kontrolle aus Managementsicht
- bessere Reflexion vergangener Projektphasen
- besserer Vergleich von Projektphasen und Projekten



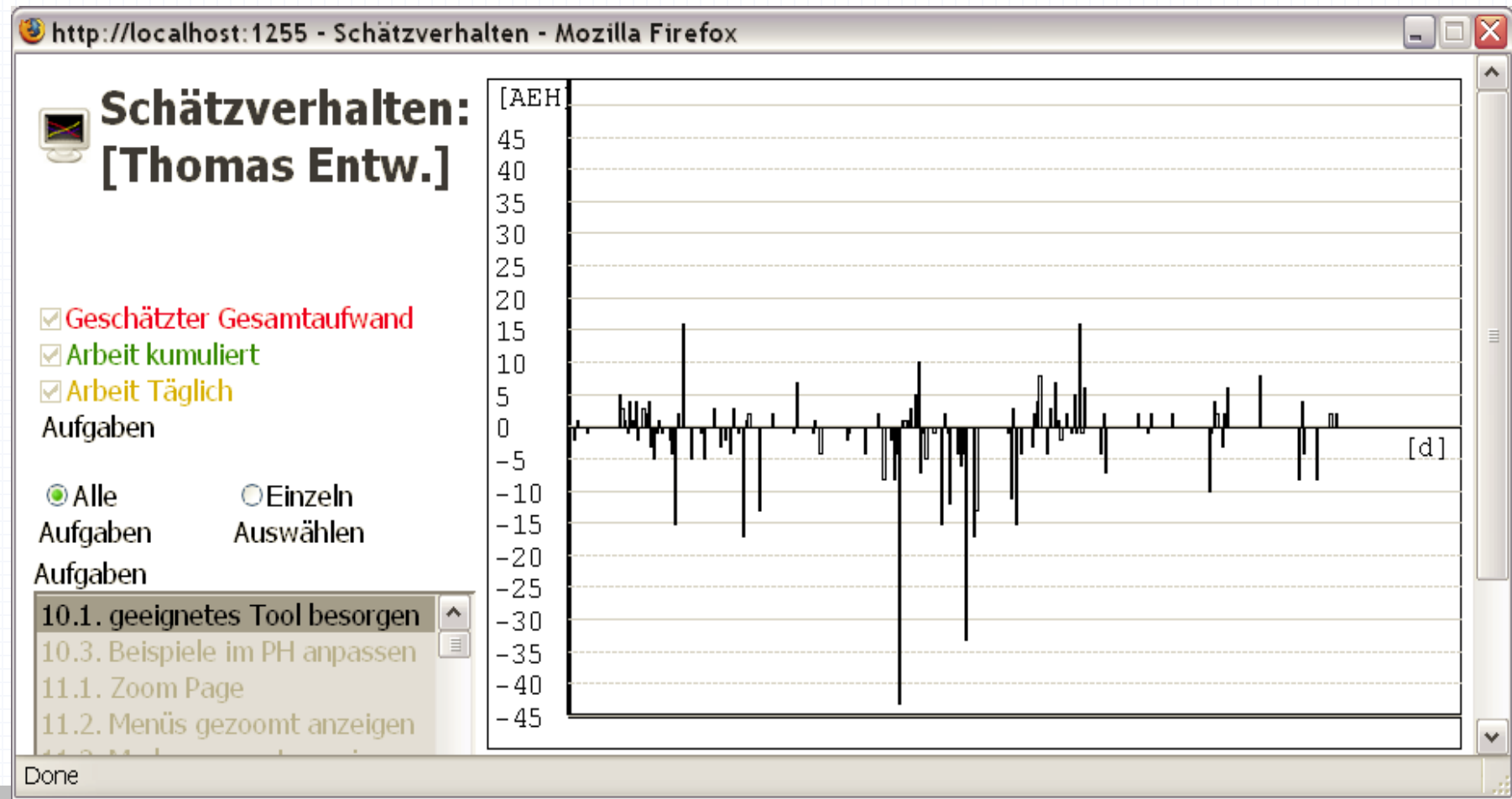
Überprüfen: Auslastung der Mitarbeiter

Sinnvoll inklusive Zuordnung zu den Aufgaben:



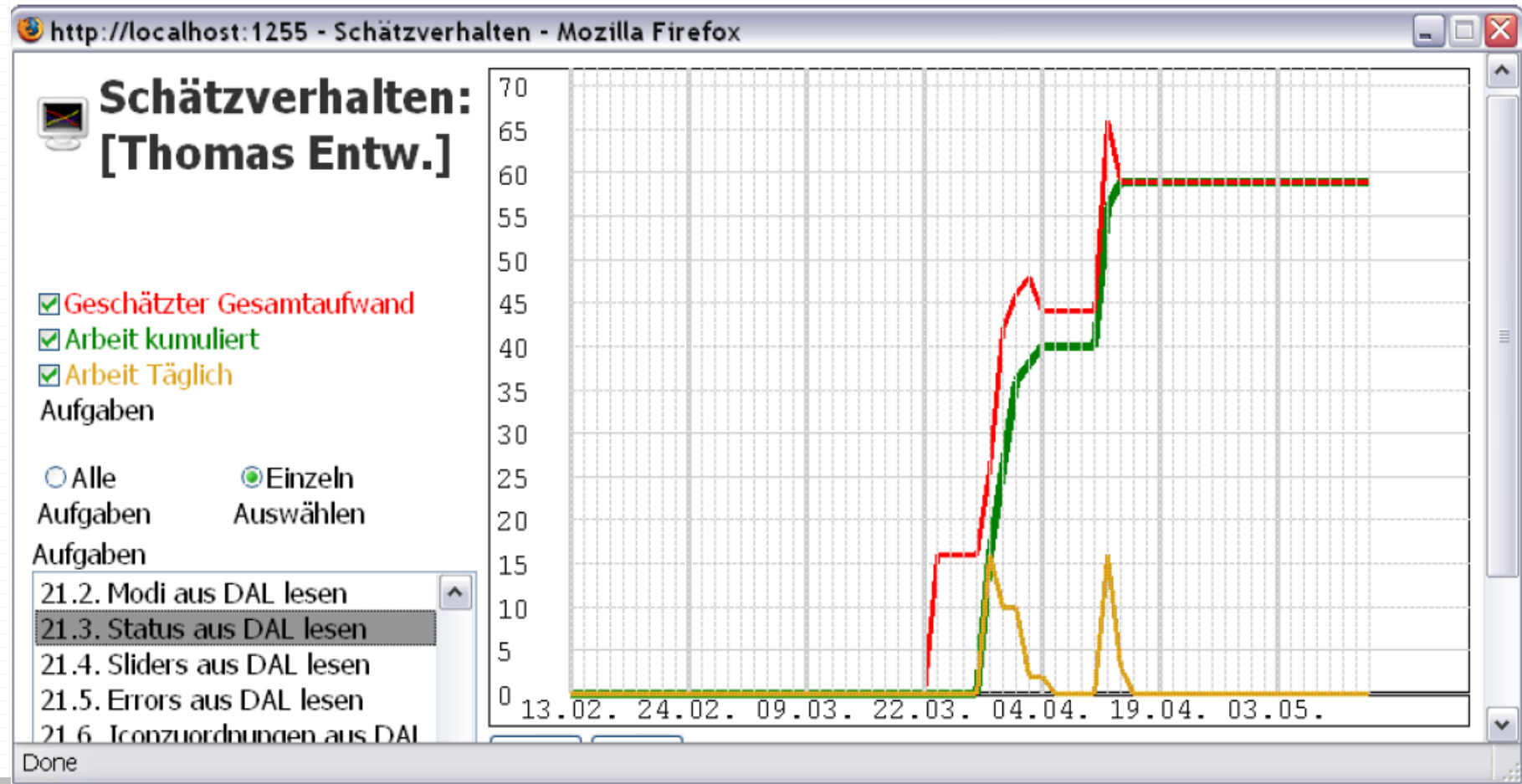
Überprüfen: Analyse des Schätzvermögens (I)

Sinnvoll nur bei laufender Soll-Planung und Ist-Erfassung!



Überprüfen: Analyse des Schätzvermögens (II)

Detaillierte Analyse pro Aufgabe:



Überprüfen: Potenzial der Prozessverbesserung

Rückbetrachtung („**Retrospektive**“) nach jeder Iteration

z. B. in Scrum nach folgenden Regeln (erst ab 2004 in Scrum ergänzt):

1. **Schaffe Sicherheit** (Wir suchen keine Fehler/Schuldige, sondern Verbesserungsmöglichkeiten.)
2. **Sammle die Fakten** (Was war wichtig?)
3. **Finde funktionierende Prozesse** (Was lief gut?)
4. **Finde nicht funktionierende Prozesse** (Was lief schlecht?)
5. **Finde die Kompetenz** (Wer kann was verbessern?)
6. **Priorisiere die Verbesserungsmöglichkeiten** (drei wichtigste *positive* und drei wichtigste *negative* Erkenntnisse; sichtbar platzieren)



PROJEKT ENGINEERING

Tipps für die agile Softwareentwicklung

Herwig Mayr

Fakultät für Informatik, Kommunikation und Medien
Fachhochschule OÖ, Hagenberg